

Parfait. On passe donc au **plan d'implémentation technique complet**, sans encore écrire le code. L'objectif est qu'à la fin, on ait un document suffisamment précis pour développer sans devoir redécider l'architecture toutes les deux semaines.

Je garde notre décision principale :

PWA/Web App d'abord → validation → iOS/Android ensuite avec Expo.

Et je garde aussi notre séparation stricte :

`listing ≠ story ≠ next_chapter ≠ order ≠ payment ≠ moderation`

Supabase reste un très bon choix pour ce produit : PostgreSQL, Auth, Storage, Realtime et Edge Functions sont intégrés, et les Edge Functions conviennent notamment aux webhooks Stripe, aux appels IA et aux opérations backend courtes.  Supa... +1

Vue d'ensemble

Je découperais le projet en **12 phases** :

Phase	Objectif
0	Valider concept, marque et hypothèses
1	Foundation technique
2	Auth, profils et sécurité
3	Marketplace Core
4	Storytelling Engine
5	Feed social & engagement
6	Next Chapter
7	Trust, Safety & Moderation
8	Messaging & Offers
9	Transactions & Stripe Connect

Phase	Objectif
10	Shipping, disputes & marketplace réelle
11	Growth, SEO, analytics & optimisation
12	iOS + Android

Mais surtout : chaque phase doit avoir un **Definition of Done** avant de passer à la suivante.

PHASE 0 — PRODUCT VALIDATION

Objectif

Ne pas passer trois mois à construire quelque chose que les gens trouvent drôle mais sur lequel personne ne veut vendre.

C'est probablement **le plus gros risque du projet**.

Nous devons tester séparément :

Hypothèse A

Les gens aiment consulter les histoires.

Hypothèse B

Les gens acceptent de raconter une histoire liée à un objet.

Hypothèse C

Ils sont réellement disposés à vendre l'objet ici plutôt que sur Vinted/eBay.

Hypothèse D

Le concept :

Sell the past. Fund what's next.

est plus attractif qu'un simple :

Sell your ex's stuff.

0.1 Branding provisoire

Créer une identité temporaire.

Nom de travail :

Formerly Yours

Mais avant de construire le branding définitif :

- recherche marques EUIPO ;
- domaines ;
- App Store ;
- Google Play ;
- TikTok ;
- Instagram ;
- X ;
- Reddit ;
- GitHub ;
- éventuellement USPTO si ambition US.

Livrable

brand-validation.md

contenant :

```
Candidate name
Trademark conflicts
Domains
Social handles
App names
Decision
```

0.2 Landing page

Une seule page très travaillée.

Sections :

```
Hero
↓
Example story
↓
How it works
↓
Fund My Next Chapter
```

↓
Example New Chapter
↓
Safety philosophy
↓
Waitlist

CTA principal :

I have something to sell

CTA secondaire :

I just want the stories

Ce détail est important parce qu'on distingue vendeurs et consommateurs de contenu.

0.3 Prototype Feed

Pas de backend complet.

Créer 20–30 exemples fictifs.

Exemple :

MacBook
Wedding ring
PS5
Coffee machine
Designer bag
Furniture
Vacation tickets
Watch
Dress
Camera

Avec histoires de catégories différentes.

Le prototype doit permettre :

- swipe ;
- réaction ;
- ouvrir annonce ;
- voir Next Chapter ;
- partager.

0.4 Validation quantitative

Événements :

```
landing_viewed
seller_cta_clicked
viewer_cta_clicked
story_viewed
story_expanded
fake_buy_clicked
waitlist_started
waitlist_completed
```

La question essentielle :

Combien de visiteurs expriment une **intention vendeur réelle** ?

Sortie Phase 0

Je n'irais en Phase 1 que si nous obtenons au moins des signaux crédibles sur :

- intérêt contenu ;
- intention vendeur ;
- compréhension immédiate du concept ;
- absence de confusion avec une plateforme de revenge content.

PHASE 1 — TECHNICAL FOUNDATION

Ici commence véritablement l'application.

1.1 Repository

Je recommande :

Monorepo

```
formerly-yours/
|
├─ apps/
|   └─ web/
|   └─ admin/
|
```

```
├─ packages/
│   ├─ database/
│   ├─ types/
│   ├─ validation/
│   ├─ domain/
│   ├─ analytics/
│   ├─ ui/
│   └─ config/
│
├─ supabase/
│   ├─ migrations/
│   ├─ functions/
│   └─ seed/
│
├─ docs/
│
├─ .github/
│   └─ workflows/
│
├─ package.json
├─ pnpm-workspace.yaml
└─ turbo.json
```

Je choisirais :

pnpm + Turborepo.

1.2 Apps

apps/web

Next.js PWA principale.

apps/admin

Back-office séparé.

Très important.

Je ne veux pas :

/admin

avec toute l'administration simplement dissimulée dans l'application publique.

La sécurité ne doit évidemment pas dépendre de cela, mais séparer le déploiement admin améliore énormément l'isolation.

1.3 Environnements

Minimum :

```
development
staging
production
```

Avec trois projets Supabase idéalement.

Development

Données de test.

Staging

Tests QA et Stripe test mode.

Production

Utilisateurs réels.

1.4 Supabase local sans Docker

Petit point important.

La documentation Supabase indique actuellement que l'exécution **locale** complète des Edge Functions via Supabase CLI nécessite Docker ou un runtime compatible.  Supab...

Mais nous ne sommes pas obligés d'utiliser cette méthode.

Si on veut éviter Docker, je préfère :

```
local Next.js
    ↓
Supabase development cloud project
```

Puis un Supabase staging distinct.

Donc :

Pas de dépendance Docker obligatoire pour notre workflow.

1.5 Variables d'environnement

Organisation :

```
.env.example  
.env.local
```

Jamais dans Git :

```
SUPABASE_SECRET_KEY  
STRIPE_SECRET_KEY  
OPENAI_API_KEY  
RESEND_API_KEY  
SENTRY_AUTH_TOKEN
```

Frontend :

```
NEXT_PUBLIC_SUPABASE_URL  
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY  
NEXT_PUBLIC_APP_URL
```

Supabase est en transition vers les nouvelles clés `publishable` et `secret`, les anciennes clés étant prévues pour dépréciation ; je construirais donc directement autour des nouvelles conventions.  Supab...

1.6 CI

À chaque pull request :

```
install  
↓  
lint  
↓  
typecheck  
↓  
unit tests  
↓  
build  
↓  
Playwright smoke tests
```

Puis preview deployment.

1.7 Branching

Simple :

```
main  
feature/*  
fix/*
```

Pas de GitFlow énorme.

`main` doit toujours être déployable.

1.8 Design system foundation

Avant les pages :

```
Typography  
Spacing  
Radius  
Shadows  
Buttons  
Inputs  
Cards  
Dialogs  
Drawers  
Tabs  
Toast  
Skeleton  
Avatar  
Badge  
Progress
```

Police moderne :

je regarderais probablement **Geist**, **Inter** ou **Manrope** selon l'identité finale.

1.9 Responsive breakpoints

Le produit est **mobile-first**.

Concevoir d'abord :

```
375 px  
390 px
```

Puis :

768
1024
1440+

Le desktop ne doit jamais ressembler à un téléphone géant étiré.

1.10 PWA Foundation

Dès maintenant :

```
manifest.webmanifest  
icons  
theme  
display: standalone  
service worker  
offline fallback  
installability
```

Même si toutes les fonctions offline ne sont pas encore présentes.

Sortie Phase 1

On doit avoir :

- ✓ repo propre
 - ✓ CI fonctionnelle
 - ✓ staging
 - ✓ Supabase connecté
 - ✓ migrations fonctionnelles
 - ✓ design system initial
 - ✓ PWA installable
 - ✓ monitoring technique initial
 - ✓ aucun secret exposé
-

PHASE 2 — AUTH + IDENTITY + SECURITY

Avant marketplace.

2.1 Auth

V1 :

- email magic link ou OTP ;
- Google ;
- Apple.

Je préfère éviter de dépendre exclusivement de passwords classiques.

2.2 User bootstrap

À chaque création Supabase Auth :

```
auth.users
  ↓ trigger
public.profiles
```

Créer automatiquement :

```
id
username nullable
display_name nullable
created_at
```

2.3 Onboarding

Étapes :

Welcome

What brings you here?

```
Sell
Browse
Both
```

Username

Country

Optional avatar

Community rules

Puis accès au produit.

2.4 Profile privacy

Jamais exposer publiquement :

- email ;
 - téléphone ;
 - adresse exacte ;
 - KYC ;
 - date de naissance ;
 - Stripe account ;
 - tax ID.
-

2.5 RLS

Toutes les tables exposées doivent avoir RLS.

Supabase recommande explicitement d'activer RLS sur les tables des schémas exposés, notamment `public` .  Supab...

Créer une suite de tests :

```
anonymous cannot edit listing
user A cannot edit user B listing
user A cannot read private conversation B/C
seller cannot edit payment status
blocked user cannot message blocker
```

2.6 Security tests

Créer une catégorie spécifique :

```
tests/security/
```

Pas seulement des tests UI.

Sortie Phase 2

Un utilisateur peut :

- ✓ créer un compte
- ✓ faire son onboarding
- ✓ modifier son profil
- ✓ se déconnecter
- ✓ supprimer/désactiver son compte
- ✓ n'accéder qu'aux données autorisées

PHASE 3 — MARKETPLACE CORE

Maintenant seulement on construit les objets.

3.1 Categories

Seed :

Fashion
Electronics
Jewelry
Luxury
Home
Gaming
Wedding
Travel
Collectibles
Other

Puis sous-catégories.

3.2 Listing draft

Créer d'abord une annonce vide :

draft

Chaque étape peut être sauvegardée.

3.3 Sell Flow

Wizard mobile.

Step 1

Photos.

Step 2

What are you selling?

Step 3

Category.

Step 4

Condition.

Step 5

Description.

Step 6

Price.

Step 7

Story — phase suivante.

3.4 Images

Pipeline :

```
client compression
↓
upload
↓
temporary private state
↓
moderation
↓
metadata stripping
↓
optimized derivative
↓
publish
```

Supprimer EXIF.

Particulièrement :

```
GPSLatitude
GPSLongitude
device metadata
```

3.5 Listing page

Route SEO :

```
/item/[slug]-[shortId]
```

Contient :

- galerie ;
 - titre ;
 - état ;
 - prix ;
 - vendeur ;
 - story preview ;
 - Next Chapter ;
 - save ;
 - share.
-

3.6 Search

Phase initiale :

PostgreSQL.

Recherche :

```
title
brand
model
description
```

Filtres :

category
condition
price
country

Pas Elasticsearch/Algolia maintenant.

3.7 Saved

Ajouter :

♥ Save

et page :

/saved

Sortie Phase 3

Utilisateur :

- ✓ crée une annonce
- ✓ charge ses photos
- ✓ sauvegarde draft
- ✓ publie
- ✓ recherche
- ✓ filtre
- ✓ sauvegarde une annonce

La marketplace fonctionne déjà sans storytelling.

PHASE 4 — STORY ENGINE

C'est ici que notre produit devient différent.

4.1 Context

Après description produit :

Okay... why is this here?

Choix :

Cheated On
Ghosted
Wedding Cancelled
Divorce
Moving Out
Terrible Gift
What Was I Thinking?
At Least They Had Taste
Peaceful Goodbye
Other

4.2 Story mode

Choisir :

Clean Break
A Little Tea
Full Story

Puis indépendamment :

Public
Limited
Anonymous

4.3 AI Story Assistant

L'utilisateur écrit ses faits.

Exemple :

He gave me this for our anniversary.
We separated six months later.
I'm moving.

Options :

Keep my words
Make it shorter
Make it witty
Make it classy
Make it playful

Je conserverais toujours :

Original user input

séparé temporairement du résultat IA pour audit/modération, avec politique de rétention appropriée.

4.4 Important : IA sans invention

Prompt système :

Never invent:
- events
- accusations
- infidelity
- crimes
- names
- motives

L'IA ne transforme que les informations explicitement fournies.

4.5 Story Page

Route :

/story/[slug]-[shortId]

optimisée partage social.

Contenu :

photo
headline
story
relationship tags
item
price
seller/chapter

4.6 Reactions

Ajouter les réactions signature.

Pas seulement `like`.

4.7 Comments

V1 :

- commentaire ;
 - une profondeur de réponse ;
 - edit ;
 - delete ;
 - report ;
 - block.
-

4.8 Sharing

Générer :

OpenGraph card.

Puis :

Copy Link
WhatsApp
X
Facebook
Reddit

Pour TikTok/Instagram :

création ultérieure d'une card image/vidéo partageable.

Sortie Phase 4

Une personne peut maintenant :

vendre un objet **et raconter pourquoi il existe sur la plateforme.**

C'est notre premier véritable MVP distinctif.

PHASE 5 — SOCIAL FEED

Maintenant le contenu devient destination.

5.1 Navigation principale

Je garderais :

Home
Discover
Sell
Inbox
You

5.2 Home

Tabs :

For You

Following

Peut-être :

Fresh

plus tard.

5.3 Ranking V1

Aucune IA complexe.

Formule approximative :

score =
0.30 recency
+ 0.20 reaction_velocity

+ 0.15 comments
+ 0.15 saves
+ 0.10 shares
+ 0.10 user_affinity

Avec safeguards contre :

- spam ;
- brigading ;
- engagement artificiel.

5.4 Feed Card

Chaque carte doit contenir suffisamment pour comprendre l'histoire sans ouvrir dix écrans.

PHOTO

€690

MacBook Pro

💔 Cheated On

"He finished his master..."

😄 18.4K 🍿 3.2K 🚩 1.1K

🌴 Bali Trip

€690 / €1,200

[View item](#)

5.5 View tracking

Éviter :

page opened = view

Je compterais une vraie impression après :

- visibilité minimale ;
 - durée minimale.
-

5.6 Follow

Suivre le vendeur / storyteller.

5.7 Realtime

Supabase Realtime peut gérer Broadcast, Presence et changements Postgres, ce qui convient à nos futures notifications, chat et certaines interactions live. Les channels privés avec autorisation RLS sont recommandés pour les scénarios authentifiés.  Supa... +1

Mais pas besoin de rendre les compteurs live à chaque réaction.

Sortie Phase 5

Nous pouvons mesurer :

- session duration ;
 - stories/session ;
 - shares ;
 - DAU/WAU ;
 - follows ;
 - retour J1/J7.
-

PHASE 6 — FUND MY NEXT CHAPTER

Deuxième grande différenciation.

6.1 Chapter creation

Créer :

Title
Goal
Target amount
Description
Visibility

Templates :

Solo Trip
New Home
Fresh Start
Study
Start a Business
Savings
Something New

6.2 Listing association

Pendant la vente :

| Where should this money go?

Choisir :

Existing chapter
Create new chapter
No chapter

6.3 Public Chapter Page

Exemple :

🌴 First Solo Trip

€840 / €1,500

3 items sold

✓ Watch €220
✓ PS5 €320
✓ Bag €300

Au début, tant que les paiements réels ne sont pas actifs, on doit clairement marquer les données comme simulations/test.

6.4 What Happened Next

Créer :

avec texte + média.

6.5 Transformation feed

Une nouvelle section pourrait être testée :

New Chapters

Mais seulement si suffisamment de contenu existe.

Sortie Phase 6

La boucle conceptuelle existe :

```
PAST
↓
OBJECT
↓
STORY
↓
SALE intention
↓
NEXT CHAPTER
```

PHASE 7 — TRUST & SAFETY

Je la fais avant l'argent réel.

7.1 Report system

Report :

```
Doxxing
Harassment
Threat
Spam
Stolen item
Scam
Counterfeit
```

Explicit content

Hate

Other

7.2 PII detection

Scan stories/comments pour :

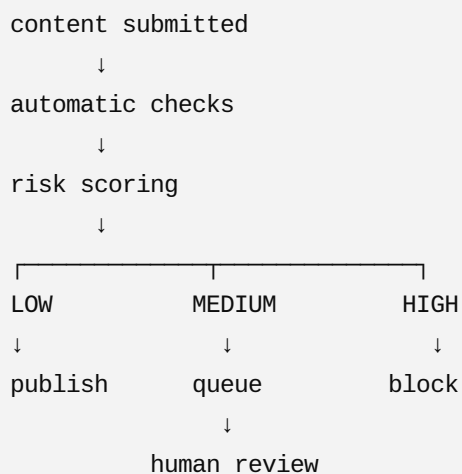
- email ;
- téléphone ;
- URL sociale ;
- adresse ;
- plaque ;
- full name patterns.

7.3 Image moderation

Détecter :

- visage de tiers ;
- screenshot ;
- document personnel ;
- téléphone ;
- coordonnées ;
- nudité ;
- contenu interdit.

7.4 Moderation pipeline



7.5 Admin App

apps/admin

Modules :

Moderation Queue
Reports
Users
Listings
Stories
Transactions later
Audit

Roles :

support
moderator
admin
super_admin

Pas simplement :

is_admin boolean

7.6 Audit

Toute action sensible :

who
what
when
entity
before
after
reason

Sortie Phase 7

Aucun paiement réel tant que :

- ✓ report
- ✓ block
- ✓ moderation
- ✓ suspension
- ✓ appeal/process basique
- ✓ audit

ne fonctionnent.

PHASE 8 — MESSAGING + OFFERS

8.1 Conversation

Une conversation autour de :

listing
buyer
seller

Une seule conversation buyer/listing idéalement.

8.2 Realtime chat

Supabase Realtime.

Channels privés.

RLS.

Typing indicator éventuellement Broadcast.

Pas besoin de Presence global.

8.3 Safety

Bloquer :

- numéros ;
- emails ;
- liens suspects ;

- tentatives paiement hors plateforme.

Afficher :

Keep payments on Formerly Yours for protection.

8.4 Offers

Acheteur :

Make Offer €610

Vendeur :

Accept
Decline
Counter

J'ajouterais finalement `counter offer` dans le modèle.

8.5 Atomic accept

Accept Offer doit exécuter transaction atomique :

offer accepted
other offers invalidated
listing reserved
checkout session created/allowed

Sortie Phase 8

Marketplace négociable fonctionnelle, encore en environnement de test.

PHASE 9 — STRIPE CONNECT

Ici seulement on introduit l'argent.

Stripe Connect est conçu précisément pour les plateformes et marketplaces où une plateforme collecte des paiements et reverse des vendeurs ; notre architecture correspond bien à ce modèle. Les responsabilités liées aux

remboursements et chargebacks doivent cependant être conçues explicitement selon le type de charge choisi.  Supab...

9.1 Seller onboarding

Avant vente réelle :

```
seller creates connected account
↓
Stripe onboarding
↓
identity/KYC
↓
payout enabled
```

9.2 Payment lifecycle

```
BUY
↓
server validates listing
↓
reserve
↓
create payment
↓
Stripe
↓
webhook
↓
payment confirmed
↓
order paid
```

Le navigateur ne confirme jamais le paiement.

9.3 Webhooks

Supabase Edge Function :

```
stripe-webhook
```

Webhook signature vérifiée.

Idempotency obligatoire.

9.4 Order State Machine

Implémenter strictement :

```
pending_payment
→ paid
→ awaiting_shipping
→ shipped
→ delivered
→ completed
```

9.5 Fees

Configuration dynamique :

```
seller price: €100
platform fee: €7
shipping: €5
buyer protection: €3
```

Ne pas hardcoder les pourcentages partout.

Créer :

```
fee_rules
```

ou configuration backend.

9.6 Ledger

Je rajouterais ici une amélioration importante à notre architecture précédente :

```
ledger_entries
```

Parce qu'avec de l'argent réel, je veux un journal financier immuable.

```
order
payment
platform fee
seller receivable
```

```
refund
chargeback
payout
chapter contribution
```

Chaque mouvement produit une écriture.

C'est beaucoup plus robuste qu'une succession de montants recalculés depuis plusieurs tables.

9.7 Financial invariants

Tests :

```
gross =
seller net
+ platform fee
+ refund
+ adjustment
```

Chaque centime doit être explicable.

Sortie Phase 9

Premier achat Stripe test.

Puis :

premier achat réel contrôlé.

PHASE 10 — SHIPPING + DISPUTES

10.1 Launch Geography

Je recommande **un seul pays au départ**.

Cela simplifie :

- shipping ;
- fiscalité ;
- protection consommateur ;

- retours ;
 - fraude ;
 - support.
-

10.2 Shipping

Provider abstraction :

```
ShippingProvider
├─ createLabel()
├─ track()
├─ cancellLabel()
└─ getStatus()
```

Ainsi nous ne couplons pas tout le produit à DHL/Colissimo/etc.

10.3 Shipment model

```
label_created
dropped_off
in_transit
delivered
exception
lost
returned
```

10.4 Dispute

Buyer :

```
Item never arrived
Different item
Major undisclosed damage
Counterfeit
Other
```

Evidence privé.

10.5 Payout delay

Ne pas forcément verser immédiatement.

Selon notre modèle Stripe/risk :

```
payment
↓
delivery
↓
short dispute window
↓
seller payout
```

À valider juridiquement et techniquement.

Sortie Phase 10

À ce stade, on possède une **vraie marketplace C2C exploitable**.

PHASE 11 — GROWTH ENGINE

Maintenant seulement on optimise croissance.

11.1 SEO

Routes publiques :

```
/story/*
/item/*
/category/*
/tag/*
/chapter/*
```

Metadata :

```
title
description
canonical
OG
Twitter card
structured data
```

11.2 Share Cards

Génération serveur :

1080 × 1920

avec :

- objet ;
- punchline ;
- prix ;
- catégorie ;
- branding.

11.3 Hall of Fame

Categories :

Story of the Week
Most Savage
Best New Beginning
At Least They Had Taste
Most Unexpected

Opt-in.

11.4 Referral

Vendeur :

Invite a friend.

Mais pas immédiatement avec système financier complexe.

11.5 Analytics

PostHog :

Acquisition

source
campaign

landing
signup

Activation

profile_completed
first_story
first_listing
first_save

Marketplace

offer
purchase
sale
time_to_sale

Social

views
reactions
comments
shares
follows

Next Chapter

chapter_created
chapter_funded
chapter_completed
update_posted

11.6 Experiments

Feature flags :

feed_algorithm_v2
ai_story_default
chapter_cta_variant
seller_fee_variant

Jamais :

JavaScript

```
if (user.id === ...)
```

11.7 Performance

Targets :

Core Web Vitals.

Optimisation :

- images ;
- server rendering ;
- caching ;
- feed pagination ;
- DB indexes ;
- query analysis.

PHASE 12 — iOS + ANDROID

Seulement maintenant.

Expo reste aujourd'hui un choix solide : EAS fournit build/signing cloud, soumission App Store/Google Play, CI/CD Workflows et OTA updates. En 2026, Expo SDK 55 utilise React Native 0.83 avec la New Architecture obligatoire, ce qui sera la base à viser plutôt qu'une architecture React Native legacy.

 Expo Docum... +1

12.1 Création de apps/mobile

```
apps/  
├─ web  
├─ admin  
└─ mobile
```

Expo Router.

12.2 Code partagé

Réutilisable :

```
packages/domain  
packages/database  
packages/validation  
packages/types  
packages/analytics
```

Pas forcément :

```
packages/ui
```

à 100 %, car web et native ont parfois besoin de composants différents.

12.3 Screens mobile

```
Feed  
Discover  
Sell  
Story  
Listing  
Chapter  
Inbox  
Orders  
Profile  
Notifications
```

12.4 Native capabilities

C'est là que le natif apporte réellement quelque chose :

Camera

prise de photo directe.

Push

meilleure UX.

Deep Links

TikTok → app directement.

Share Extension

Envoyer potentiellement du contenu vers Formerly Yours.

Biometrics

protection compte éventuellement.

Better gestures

feed/swipe.

12.5 EAS

Pipelines :

development
preview
production

Expo EAS permet actuellement Build, Submit, Workflows et Update pour ce cycle de livraison.  Expo Docum... +1

Mais quand déclencher Phase 12 ?

Pas :

quand le web est terminé.

Mais lorsque les données montrent que le natif apporte de la valeur.

Je définirais trois signaux :

Signal 1

70 % du trafic est mobile.

Probable.

Signal 2

Rétention suffisamment bonne.

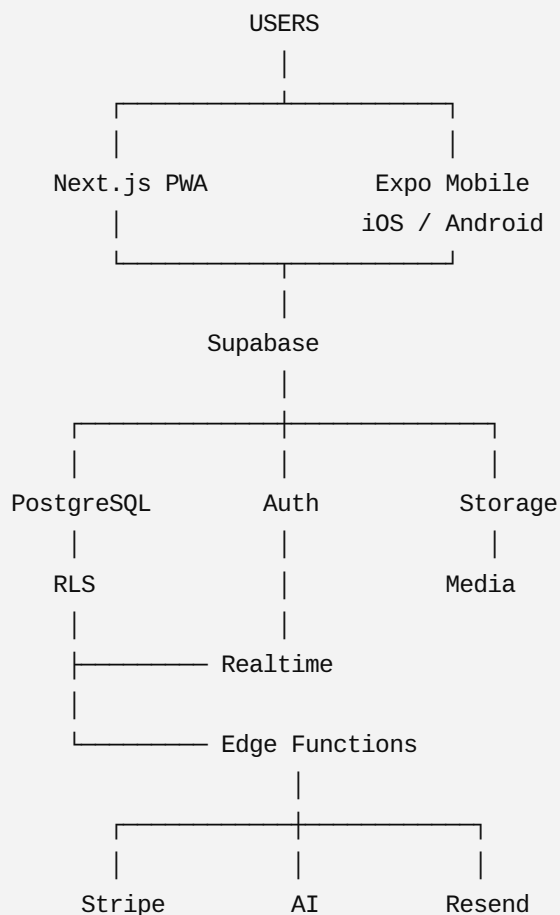
Signal 3

Les limites web deviennent concrètes :

- notifications ;
- caméra ;
- partage ;
- navigation fréquente ;
- rétention.

Alors native.

Architecture finale



Ordre concret de développement

Si demain on ouvre GitHub, l'ordre que je donnerais au développeur serait :

Sprint/Foundation

FY-001 Repository

FY-002 Next.js app

FY-003 Supabase projects

FY-004 Environment system
FY-005 CI
FY-006 Design tokens
FY-007 PWA manifest
FY-008 Analytics
FY-009 Sentry

Identity

FY-010 Auth
FY-011 Profile schema
FY-012 Profile trigger
FY-013 RLS profiles
FY-014 Onboarding
FY-015 Account settings

Marketplace

FY-020 Categories
FY-021 Listing schema
FY-022 Listing RLS
FY-023 Draft listing
FY-024 Images
FY-025 Sell wizard
FY-026 Listing page
FY-027 Search
FY-028 Saved listings

Stories

FY-030 Story schema
FY-031 Relationship contexts
FY-032 Story editor
FY-033 Story modes
FY-034 Visibility
FY-035 AI rewrite
FY-036 Moderation
FY-037 Story page
FY-038 Reactions
FY-039 Comments
FY-040 Sharing

Social

FY-050 Feed query
FY-051 Feed UI
FY-052 View events

FY-053 Ranking V1
FY-054 Following
FY-055 Discover

Next Chapter

FY-060 Chapter schema
FY-061 Create chapter
FY-062 Chapter page
FY-063 Attach listing
FY-064 Progress model
FY-065 Updates

Safety

FY-070 Reports
FY-071 Blocks
FY-072 Admin roles
FY-073 Moderation queue
FY-074 PII detection
FY-075 Image moderation
FY-076 Audit
FY-077 Account suspension

Marketplace transactions

FY-080 Conversations
FY-081 Messages
FY-082 Realtime
FY-083 Offers
FY-084 Reserve listing
FY-085 Stripe Connect
FY-086 Checkout
FY-087 Stripe webhooks
FY-088 Orders
FY-089 Ledger
FY-090 Refunds

Logistics

FY-100 Shipping
FY-101 Tracking
FY-102 Delivery
FY-103 Disputes
FY-104 Seller payout

Growth

FY-110 SEO
FY-111 OG cards
FY-112 Social share cards
FY-113 Hall of Fame
FY-114 Referrals
FY-115 Experiments

Native

FY-200 Expo foundation
FY-201 Auth mobile
FY-202 Feed
FY-203 Sell flow
FY-204 Camera
FY-205 Inbox
FY-206 Push
FY-207 Deep links
FY-208 Orders
FY-209 iOS release
FY-210 Android release

Trois milestones que je fixerais

MILESTONE A — The Story Product

Phases 0-5.

À ce moment :

les gens peuvent venir, créer une histoire autour d'un objet et consommer le feed.

Pas encore de paiement réel.

On valide :

est-ce intéressant ?

MILESTONE B — The Transformation Product

Phase 6-7.

On ajoute :

Fund My Next Chapter + What Happened Next + Safety.

On valide :

avons-nous une identité émotionnelle différente d'une marketplace classique ?

MILESTONE C — The Marketplace

Phases 8-10.

Messaging + offers + payment + shipping.

On valide :

les gens achètent-ils réellement ici ?

Une chose que je changerais par rapport à beaucoup de projets startup

Je ne construirais **pas toute la marketplace avant de lancer quelque chose.**

Je publierais probablement une première beta après Phase 5 :

Accounts
Listings
Stories
Feed
Reactions
Comments
Shares

Et le bouton achat pourrait au départ être :

I'm interested

plutôt que du paiement complet.

Pourquoi ?

Parce que notre avantage concurrentiel n'est pas Stripe.

Vinted sait déjà gérer des paiements.

Notre hypothèse différenciante est :

un objet avec une histoire crée suffisamment d'attention et de connexion pour générer une nouvelle forme de marketplace.

C'est cela qu'il faut tester **en premier**.

Si ça fonctionne, ajouter Stripe, shipping et l'application native devient rationnel.

Si ça ne fonctionne pas, on aura découvert le problème avant d'avoir construit un système financier, des disputes et trois applications.

Et avec ce plan, nous avons désormais une distinction très nette entre **architecture, planification** et **implémentation réelle**. La prochaine fois que nous passerons à l'implémentation, on pourra démarrer directement par **FY-001** et progresser ticket par ticket sans refaire les choix structurants à chaque étape.